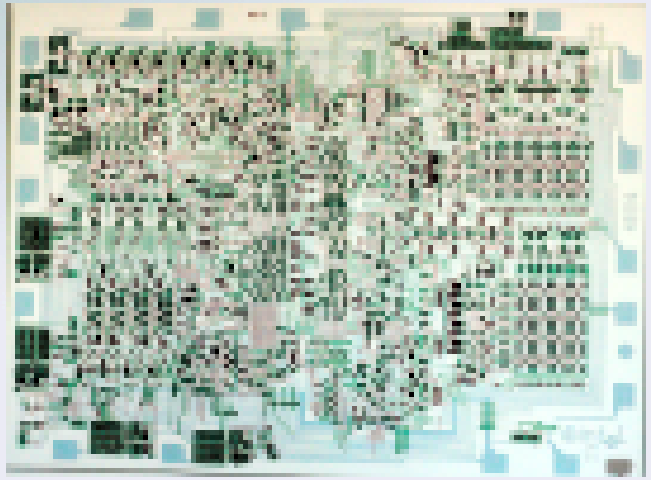


Lecture 6a: Programming the LC-3 in Assembly Language

CSCI11: **Computer Architecture and Organization**

AI Stuff

Waiting for code

- Compile times => Agent times
- Siberian Reset

AI Stuff - Naval

Noteworthy what he thinks about the role of software engineers in the future: “Does this mean that traditional software engineering is dead? Absolutely not. Software engineers—even the ones who are not necessarily tuning or training AI models—these are now among the most leveraged people on earth. [...] But software engineers still have two massive advantages on you. First, they think in code, so they actually know what’s going on underneath. And all abstractions are leaky. [...] So if you want to build a well-architected application, if you want to be able to even specify a well-architected application, if you want to be able to make it run at high performance, if you want it to do its best, if you want to catch the bugs early, then you’re going to want to have a software engineering background.”

AI Stuff - Marc Andreessen

“If the goal is to be a mediocre coder, then just let the AI do it. It’s fine. The AI is going to be perfectly good in generating infinite amounts of mediocre code. No problem. It’s all good. If the goal is, ‘I want to be one of the best software people in the world, and I want to build new software products and technologies that really matter,’ then yeah, you, 100%, want to still... You want to go all the way down. You want your skillset to go all the way down to the assembly, to assembly and machine code. You want to understand every layer of the stack.

AI Stuff - Daniel Jeffries

The jobs apocalypse is the Population Bomb of our time.

Instead we're seeing more hiring in the job most affected by AI: programming. That should have been clear and obvious to anyone with basic economics understanding and good handle on the history of technology but it's sadly lacking today.

AI Stuff - Simon Willisonn

For example: Hoard things you know how to do. “The key idea here is that coding agents mean we only ever need to figure out a useful trick once. If that trick is then documented somewhere with a working code example our agents can consult that example and use it to solve any similar shaped project in the future.” Wish I was a hoarder.

Readings

- *Chapter 7* Patt and Patel: Introduction to Computing Systems...
- *Guide to Using LC-3 Tools* - *Instructor Repo*
- *Appendix A* Patt and Patel: Introduction to Computing Systems... - *Instructor Repo*
- *LC3 Cookbook* - *Instructor Repo*

Syllabus Updates

Date	Week	Content
3/9	6	Cookbook and Programming
3/16	7	The Stack and Programming
3/23	-	**Spring Break**
3/30	8	SMC/RPN and Programming
4/5	9	TRAPS and Complete SMC
4/12	10	Test 2

LC3 Cookbook

A book or text which demonstrates how to solve specific problems. Feel free to use the code altered or unaltered in your programs.

Assembly Process

- *Assembler*: Converts assembly language file (.asm) into an executable file (.obj) … for the LC-3 simulator in our case.
- **First Pass:**
  - scan program file
  - find all labels and calculate the corresponding addresses; *symbol table*
- **Second Pass:**
  - convert instructions to machine language, using information from *symbol table*

Symbol Table for *equal_test.asm*

.SYMBOL

ADDR | EXT | LABEL

3009 | 0 | EQUAL

300B | 0 | DONE

300C | 0 | A

300D | 0 | B

300E | 0 | EQ_MSG

3020 | 0 | NOT_EQ

Style Guidelines

1. Provide program header comments, **see Cookbook code examples**
2. **Must include** a .ORIG directive at start of program and .END at end of program
   The *.ORIG x[address]* tells assembler to load the program starting at that address; as in *.ORIG x3000* load the program at *x3000*
3. **Lineup** labels, opcode, operands, and comments in same column for each line.
4. Use comments to explain what each register does.
5. Give explanatory comment for most instructions.
6. Use meaningful symbolic names.
7. Provide comments between program sections.

Your grade will depend on style as well as code effectiveness

LC-3 Memory

- The LC-3 has:
  - 16-bit address registers
  - 16-bit data registers
- Which means it can handle 65,535 16-bit words or it has 128kB of memory

LC-3 Memory Map

System Areas (0x0000 - 0x2FFF):

User Space (0x3000 - 0xBFFF):

- **User code and stack**: x3000 - xBFFF
- Use `.ORIG x3000` to place your code in this space
- **If you find your PC (program counter) isn't in this range, you've got a bug in your program**

Device and I/O Areas (0xC000 - 0xFFFF):

Accessing Data

- A *variable* is a memory location name, which is expected to change
- A *constant* is a memory location name, which is expected to stay the same
- LC-3 doesn't differentiate
- Use meaningful names, and try to use names and not values
- LC-3 uses 2's complement (*signed*) numbers, xffff == -1 and sets the N flag on load

Declaring a Variables and Constants

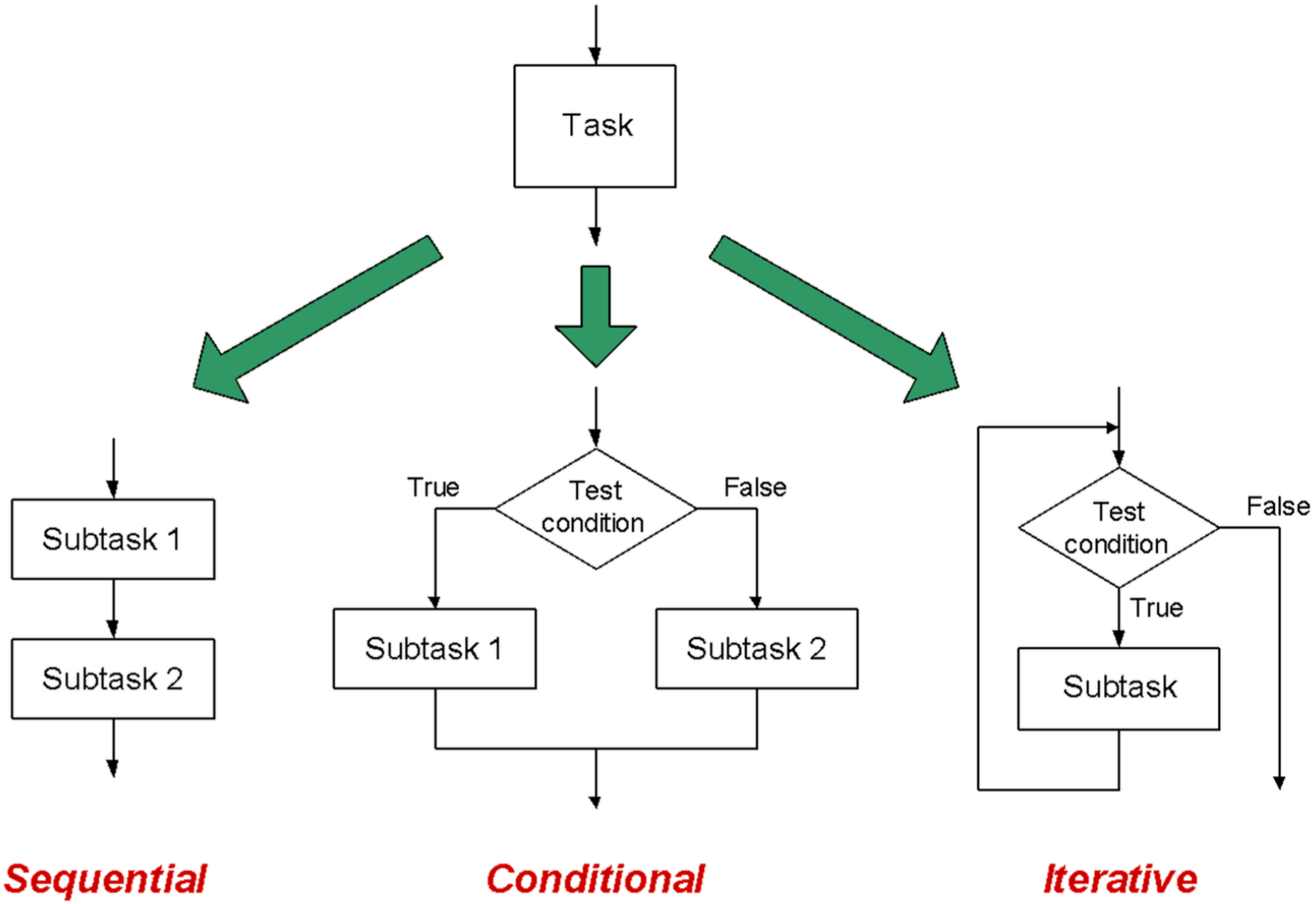
LABEL*	Directive	Parameter	Description
VARIABLE	.BLKW	n for n=1	reserves a location called VARIABLE and initializes to 0
VARIABLE	.FILL	n	reserves a location called VARIABLE and initializes to n, this could also be a constant
ARRAY	.BLKW	n for n> 1	reserves n locations, initialized to 0, intended to be used as a block (or array)

*LABEL names are your choice

Developing the Program

Three basic ways to decompose a task:

1. Sequential
2. Conditional
3. Iterative


Task
Subtask 1
Subtask 2
True
Test condition
False
Subtask 1
Subtask 2
Test condition
False
True
Subtask
Sequential
Conditional
Iterative

Sequential

- Solve the problem, **step by step**.

Coding Instruction

Instructions naturally flow from one to the next, so no special instruction needed to go from one sequential subtask to the next.

Conditional - until *"something"* happens

- Solve the problem, **based on condition**
  - if condition is true, do Subtask 1;
  - else, do Subtask 2.

Coding Instruction

See below.

Iterative - until a *"value"* has been achieved

- Solve the problem, by *counting*
  - by performing subtask over and over
  - as long as the test condition is true.

Coding Instruction

See below.

LC-3 Condition Codes (All of them)

- Each time one of the eight registers is written to, a flag will be set, based on the result of the operation

Result	*Z*	*P*	*N*	BR command
= 0	**set**	cleared	cleared	BRz
< 0	cleared	cleared	**set**	BRn
> 0	cleared	**set**	cleared	BRp
!= 0	**NOT set**	*N/A*	*N/A*	BRnp
>= 0	*N/A*	**NOT set**	*N/A*	BRzp
<= 0	*N/A*	*N/A*	**NOT set**	BRzn
always	n/a	n/a	n/a	BRnzp

Confirm Flag Operations

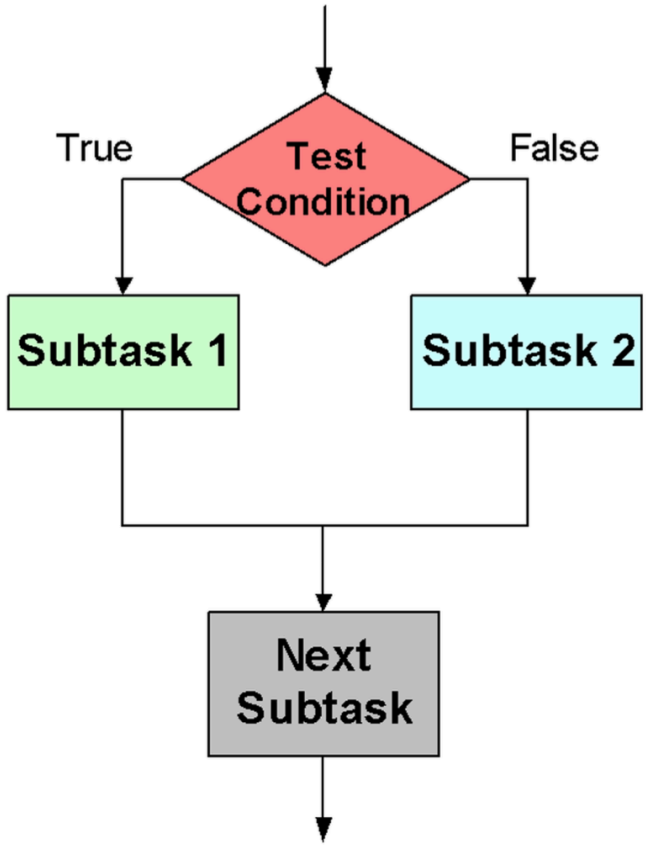
```
; test_flags
; Demonstrates:
; how values loaded in a register affect CC flags
; load R1, single-step then observe CC flags

; Registers:
; R1 - loads with different values

        .ORIG x3000 ; program starts at address x3000
        LD R1, P7f  ; 15th bit is 0
        LD R1, Pff  ; 15th bit is 1
        LD R1, N-1  ; let assembler create number
        LD R1, ZERO ; check zero as well
DONE    HALT        ; halt program
P7f     .FILL x7fff ; 15th bit is 0
Pff     .FILL xffff ; 15th bit is 1
N-1     .FILL #-1   ; let assembler create number
ZERO    .FILL x0    ; 0
        .END        ; end of program
```

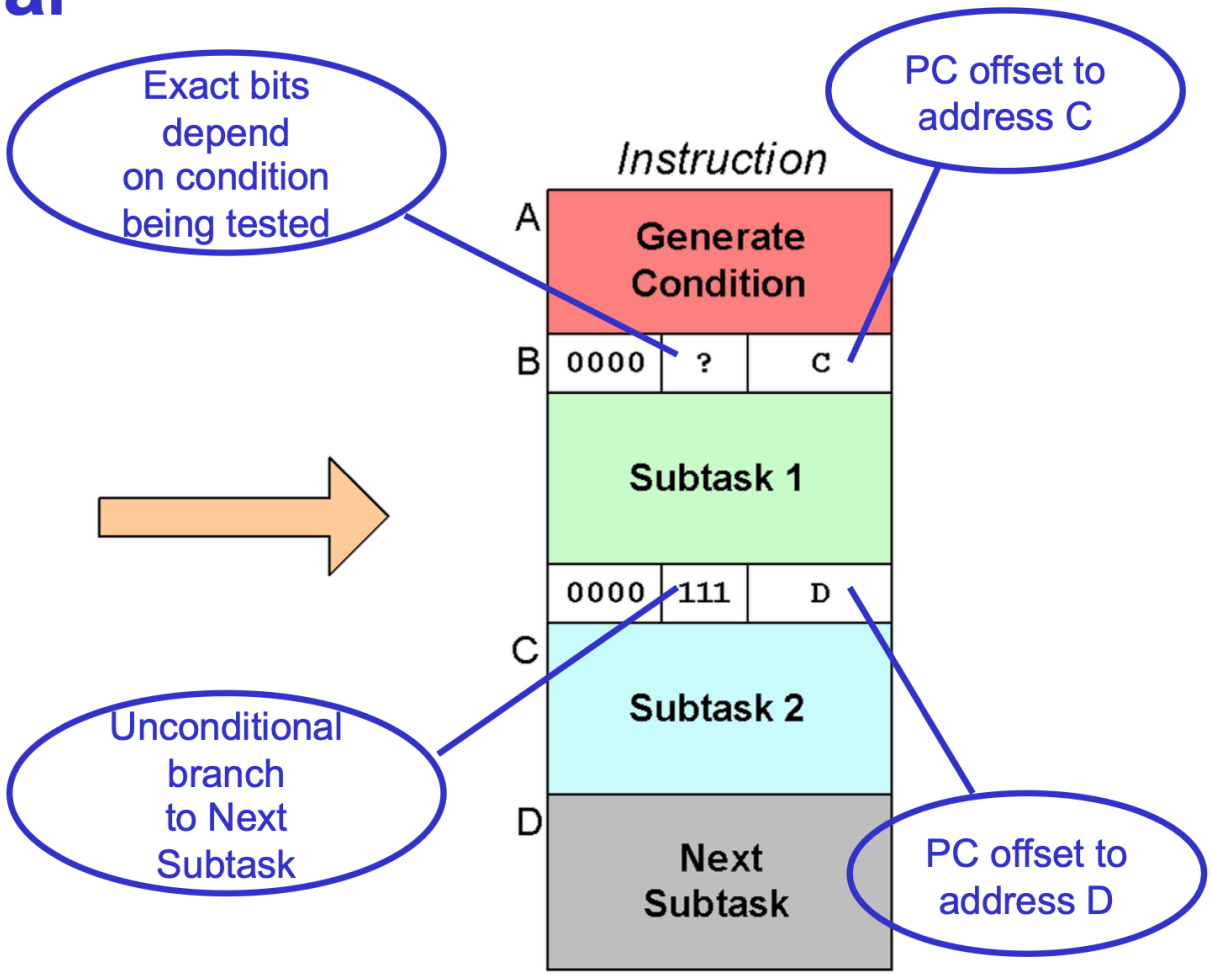
Conditional and *Iterative* Coding Instruction

Example:

- Condition: “Is R0 = R1?”
- Code: Subtract R1 from R0; **if equal**, Z bit will be set.
- Then use `BRz` instruction to transfer control if *R1 == R2*
- Or use `BRnp` instruction to *R1 != R2*
- **Note:** BRnzp results in “always branch”, and might be required in the event of skipping conditional code

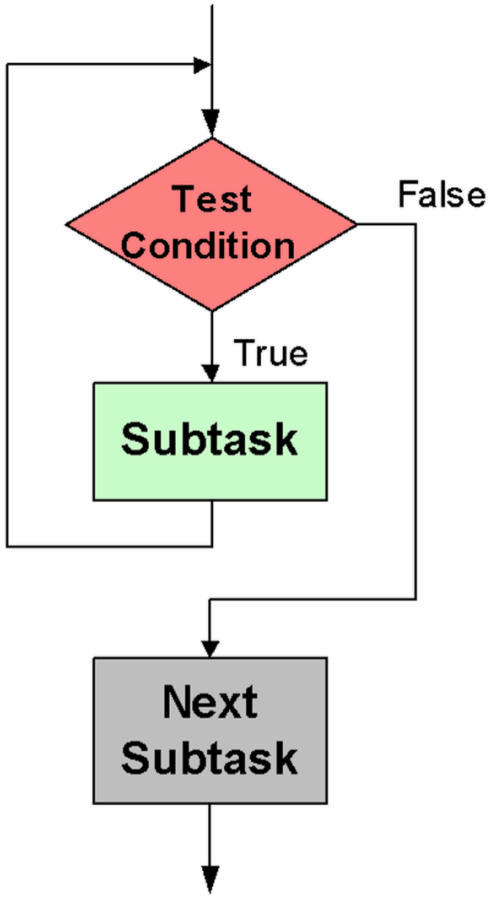
Code for Conditional

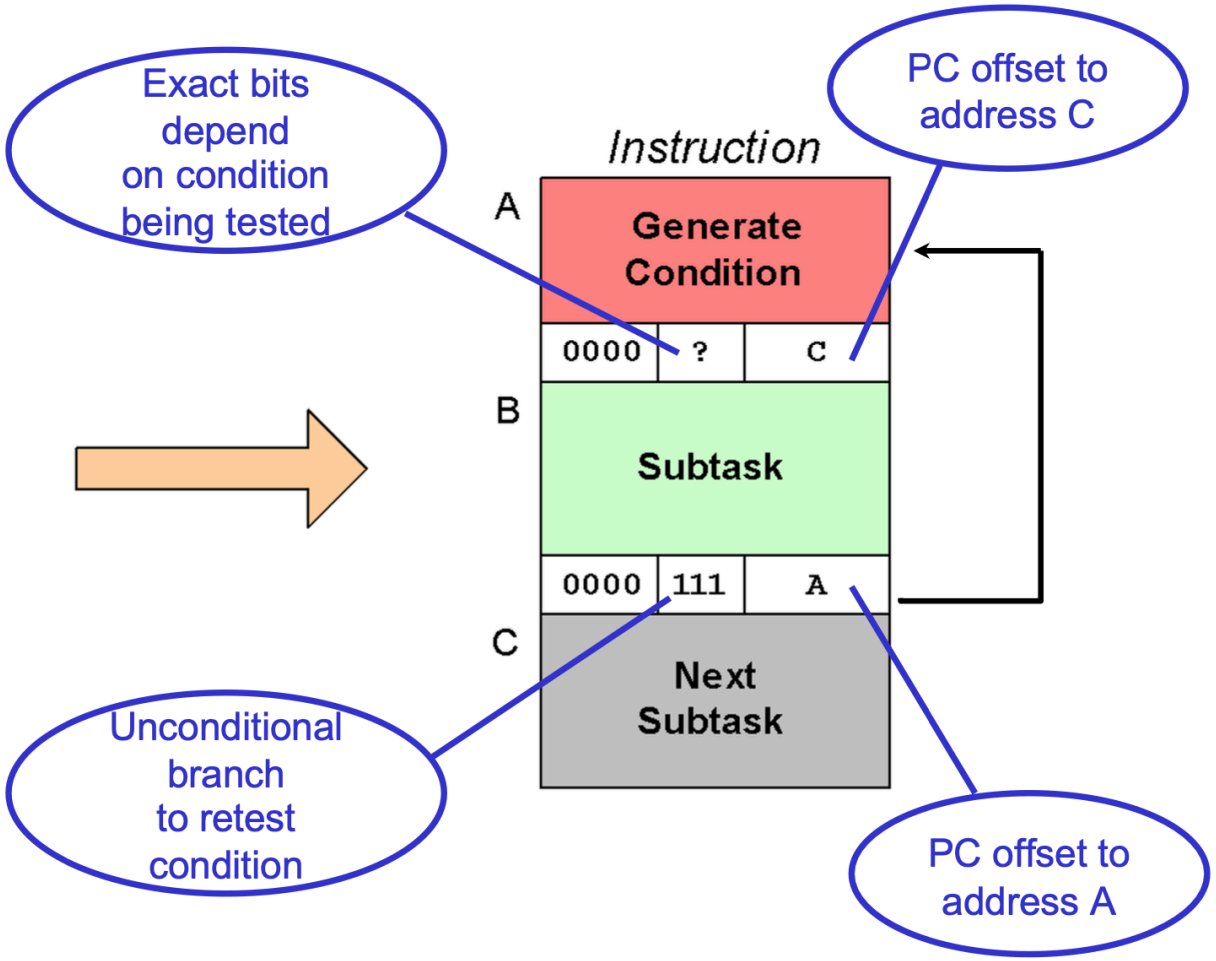




Assuming all addresses are close enough that PC-relative branch can be used.

Code for Iteration





Assuming all addresses are on the same page.

ASCII TABLE

Decimal	Hexadecimal	Binary	Octal	Char
0	0	0	0	*[NULL]*
1	1	1	1	*[START OF HEADING]*
2	2	10	2	*[START OF TEXT]*
3	3	11	3	*[END OF TEXT]*
4	4	100	4	*[END OF TRANSMISSION]*
5	5	101	5	*[ENQUIRY]*
6	6	110	6	*[ACKNOWLEDGE]*
7	7	111	7	*[BELL]*
8	8	1000	10	*[BACKSPACE]*
9	9	1001	11	*[HORIZONTAL TAB]*
10	A	1010	12	*[LINE FEED]*
11	B	1011	13	*[VERTICAL TAB]*
12	C	1100	14	*[FORM FEED]*
13	D	1101	15	*[CARRIAGE RETURN]*
14	E	1110	16	*[SHIFT OUT]*
15	F	1111	17	*[SHIFT IN]*
16	10	10000	20	*[DATA LINK ESCAPE]*
17	11	10001	21	*[DEVICE CONTROL 1]*
18	12	10010	22	*[DEVICE CONTROL 2]*
19	13	10011	23	*[DEVICE CONTROL 3]*
20	14	10100	24	*[DEVICE CONTROL 4]*
21	15	10101	25	*[NEGATIVE ACKNOWLEDGE]*
22	16	10110	26	*[SYNCHRONOUS IDLE]*
23	17	10111	27	*[END OF TRANS. BLOCK]*
24	18	11000	30	*[CANCEL]*
25	19	11001	31	*[END OF MEDIUM]*
26	1A	11010	32	*[SUBSTITUTE]*
27	1B	11011	33	*[ESCAPE]*
28	1C	11100	34	*[FILE SEPARATOR]*
29	1D	11101	35	*[GROUP SEPARATOR]*
30	1E	11110	36	*[RECORD SEPARATOR]*
31	1F	11111	37	*[UNIT SEPARATOR]*
32	20	100000	40	*[SPACE]*
33	21	100001	41	**!**
34	22	100010	42	**"**
35	23	100011	43	**#**
36	24	100100	44	**$**
37	25	100101	45	**%**
38	26	100110	46	**&**
39	27	100111	47	**'**
40	28	101000	50	**(**
41	29	101001	51	**)**
42	2A	101010	52	*****
43	2B	101011	53	**+**
44	2C	101100	54	**,**
45	2D	101101	55	**-**
46	2E	101110	56	**.**
47	2F	101111	57	**/**
48	30	110000	60	**0**
49	31	110001	61	**1**
50	32	110010	62	**2**
51	33	110011	63	**3**
52	34	110100	64	**4**
53	35	110101	65	**5**
54	36	110110	66	**6**
55	37	110111	67	**7**
56	38	111000	70	**8**
57	39	111001	71	**9**
58	3A	111010	72	**:**
59	3B	111011	73	**;**
60	3C	111100	74	**<**
61	3D	111101	75	**=**
62	3E	111110	76	**>**
63	3F	111111	77	**?**
64	40	1000000	100	**@**
65	41	1000001	101	**A**
66	42	1000010	102	**B**
67	43	1000011	103	**C**
68	44	1000100	104	**D**
69	45	1000101	105	**E**
70	46	1000110	106	**F**
71	47	1000111	107	**G**
72	48	1001000	110	**H**
73	49	1001001	111	**I**
74	4A	1001010	112	**J**
75	4B	1001011	113	**K**
76	4C	1001100	114	**L**
77	4D	1001101	115	**M**
78	4E	1001110	116	**N**
79	4F	1001111	117	**O**
80	50	1010000	120	**P**
81	51	1010001	121	**Q**
82	52	1010010	122	**R**
83	53	1010011	123	**S**
84	54	1010100	124	**T**
85	55	1010101	125	**U**
86	56	1010110	126	**V**
87	57	1010111	127	**W**
88	58	1011000	130	**X**
89	59	1011001	131	**Y**
90	5A	1011010	132	**Z**
91	5B	1011011	133	**[**
92	5C	1011100	134	****
93	5D	1011101	135	**]**
94	5E	1011110	136	**^**
95	5F	1011111	137	**_**
96	60	1100000	140	**`**
97	61	1100001	141	**a**
98	62	1100010	142	**b**
99	63	1100011	143	**c**
100	64	1100100	144	**d**
101	65	1100101	145	**e**
102	66	1100110	146	**f**
103	67	1100111	147	**g**
104	68	1101000	150	**h**
105	69	1101001	151	**i**
106	6A	1101010	152	**j**
107	6B	1101011	153	**k**
108	6C	1101100	154	**l**
109	6D	1101101	155	**m**
110	6E	1101110	156	**n**
111	6F	1101111	157	**o**
112	70	1110000	160	**p**
113	71	1110001	161	**q**
114	72	1110010	162	**r**
115	73	1110011	163	**s**
116	74	1110100	164	**t**
117	75	1110101	165	**u**
118	76	1110110	166	**v**
119	77	1110111	167	**w**
120	78	1111000	170	**x**
121	79	1111001	171	**y**
122	7A	1111010	172	**z**
123	7B	1111011	173	**{**
124	7C	1111100	174	**|**
125	7D	1111101	175	**}**
126	7E	1111110	176	**~**
127	7F	1111111	177	*[DEL]*

Review Cookbook